

Code has a memory.

Find where behavior changed.
Keep the evidence. Watch what comes next.

CPU ONLY

JAVASCRIPT + JSX

SOURCE + HISTORY

Vellore Institute of Technology (VIT University)

Advay Vivek / 24BCT0136

advay.vivek2024@vitstudent.ac.in

Kaavish Gogia / 24BCT0103

kaavish.gogia2024@vitstudent.ac.in

CodeStrata

v1 / HOLDS

connectDevice`await policyCheck();`

v2 / AWAIT REMOVED

pairDevice`policyCheck();`

v3 / RESTORED

pairDevice`await policyCheck();`

The right code can be in the wrong version.

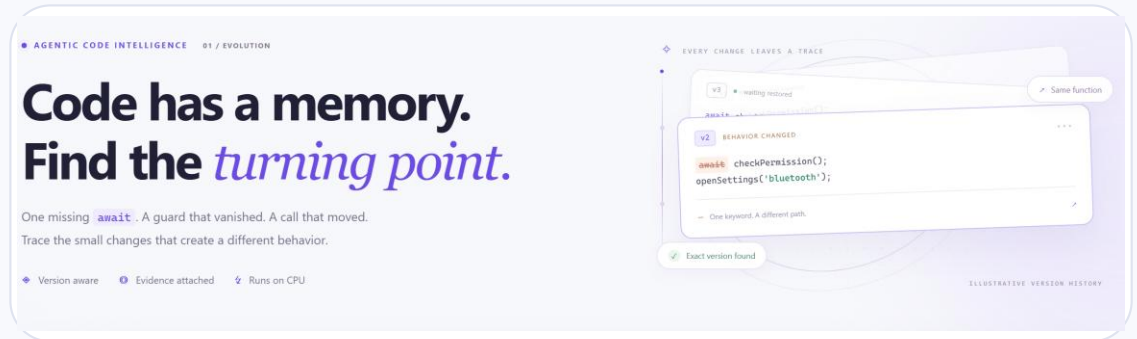
Names move, behavior shifts, and a near-identical result can hide the change that matters.

PROBLEM STATEMENT

“Where did device pairing stop waiting for policyCheck?”

Search may find a familiar name in the wrong version. One removed `await` can change the answer while the surrounding code stays similar.

A useful answer needs the version, the exact source, and a neighboring counterexample.



THE ACTUAL WORKSPACE

Ask a behavior question. Inspect evidence across versions. Save the result as a rule.

RETRIEVE → EXPLAIN → WATCH

One token can change the answer.

The gap we target: connecting a behavior question to the correct implementation in history.

01 Names can change

A moved or renamed function is harder to follow by symbol alone.

02 Similar code can disagree

Two near-identical versions may differ in their direct await or call order.

03 A diff still needs context

Reviewers need the source, version, and evidence behind an answer.

SAME CALL • DIFFERENT VERSION

v1 / devices/connect.js

```
await policyCheck(device);
```

v2 / flows/pair.js

```
policyCheck(device);
```

A removed direct await is inspectable evidence,
not proof of a runtime race.

A complete evidence loop for code history.

A question becomes an inspectable finding, then a rule the reviewer can run again.

01

Find the right snapshot

Search across indexed versions, even when a function moved or changed its name.

02

Explain the difference

Show source lines, static evidence, the linked version, and a counterexample.

03

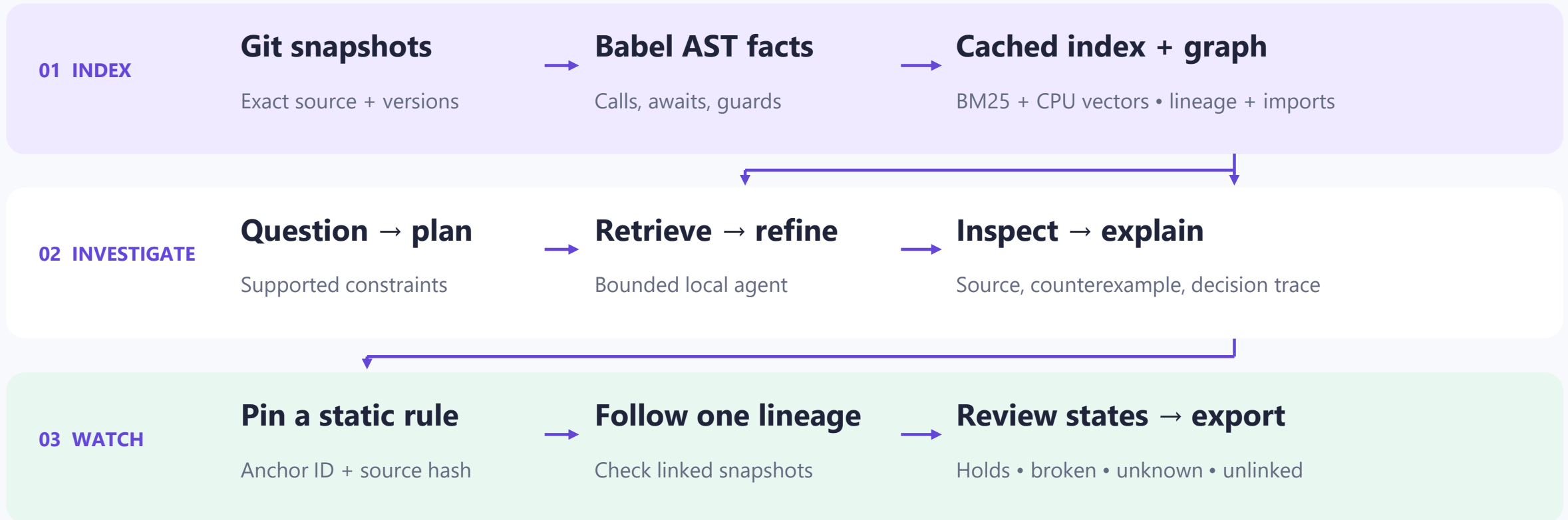
Keep the finding

Pin a structural rule and recheck the same conservative lineage later.

QUESTION → VERSION-AWARE SEARCH → SOURCE-BACKED ANSWER → BEHAVIOR WATCH

From Git history to behavior evidence.

Cached analysis feeds a bounded search policy and a repeatable watch workflow.



Read the source once. Reuse the evidence.

The index preserves executable facts separately from the text used for retrieval.

1

Load snapshots

Read the working tree or selected Git history and retain version, path, and exact source.

2

Parse functions

Babel records calls, direct awaits, guards, literals, imports, and source locations.

3

Build candidates

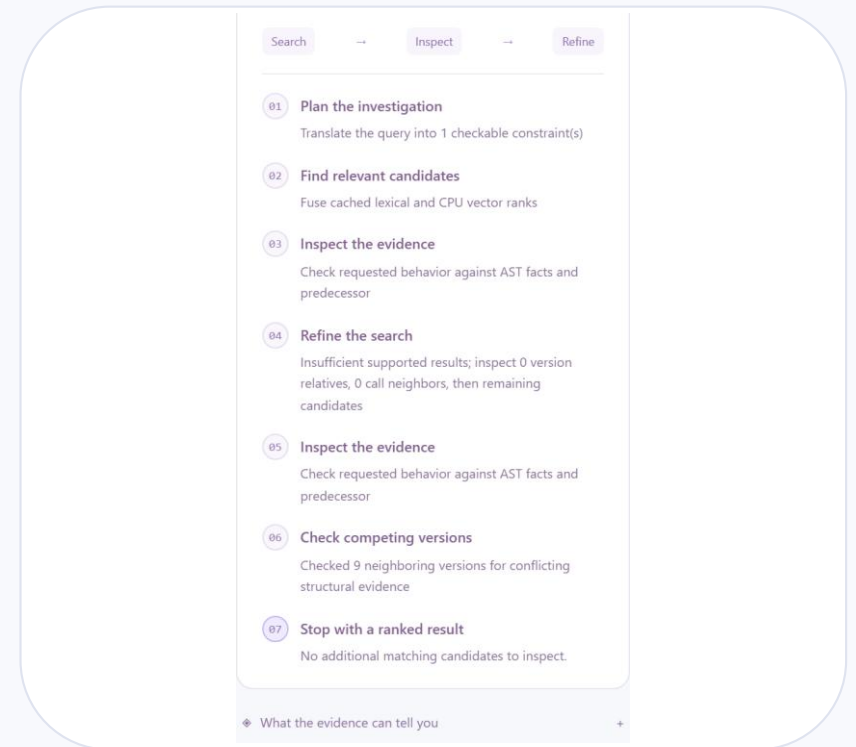
Cache lexical postings, CPU vectors, import links, and conservative lineage.

Trust boundary: repository code is read and analyzed; it is never executed during indexing.

The agent decides what to inspect next.

It records a real search policy instead of generating an unsupported answer.

1	PLAN	Parse the supported structural question; warn when phrasing exceeds the analyzer.
2	RETRIEVE	Fuse cached BM25 and CPU vector ranks to form a candidate list.
3	INSPECT	Test candidates for support, contradiction, or uncertainty from source facts.
4	EXPAND	Follow linked versions or resolved imports when the first pass is insufficient.
5	STOP	Return evidence when sufficient, exhausted, canceled, or over budget.



Budget: 3 rounds • 120 candidates.

An answer you can inspect.

The agent retrieves a version, checks its static facts, then shows the source that supports and contradicts it.

01 pairDevice
flows/pair.js:3-6 · v2 ✓ Pattern supported

Watch behavior

v1 connectDevice
First linked snapshot

v2 pairDevice
removed await · moved file · renamed symbol

v3 pairDevice
added await

✓ RETRIEVED - v2

```
3 async function pairDevice(device) {  
4   policyCheck(device);  
5   return openDevice(device, 'settings://pair-device');  
6 }
```

↔ COUNTEREXAMPLE - v1

```
3 async function connectDevice(device) {  
4   await policyCheck(device);  
5   return openDevice(device, 'settings://pair-device');  
6 }
```

Every matching direct invocation is awaited

✓ Await removed relative to v1
↗ Inferred rename / move · unique structural match. Inspect both versions to confirm identity.

CONNECTED CODE [→ policyCheck](#) [→ openDevice](#)

v1 → v2 [removed await · policyCheck](#) [moved file](#) [renamed symbol](#)

[+ Compare the preceding version - v1](#)

[+ View ranking signals](#)

1 Find pairDevice @ v2

Open exact source and line numbers.

2 Compare connectDevice @ v1

Inspect the inferred rename / move.

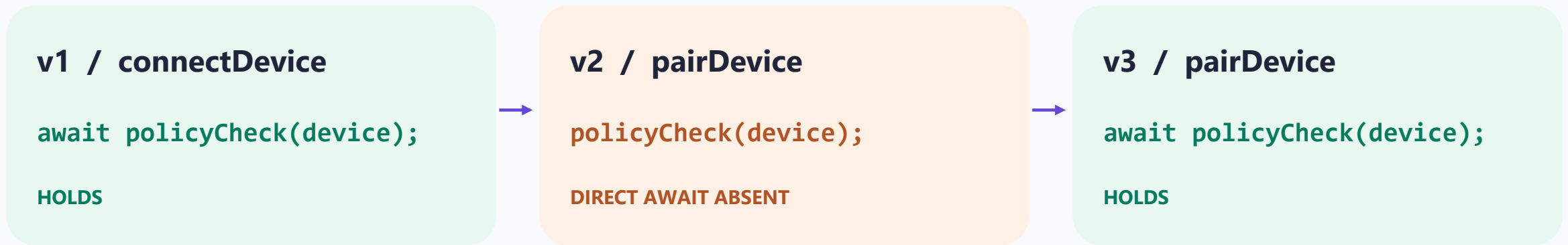
3 Follow policyCheck

Navigate the resolved imported helper.

VIDEO URL: [TEAM TO ADD AFTER RECORDING · 5 MIN MAX]

Follow a refactor without inventing identity.

A rename or move is linked only when the structural match is strong and unambiguous.



How links are proposed

Same-file identity first; unmatched functions use normalized AST structure and call-target overlap.

When we refuse a link

Competing matches, copies, deletion gaps, and major refactors remain unresolved rather than guessed.

How the investigation actually runs.

A deterministic local policy records its search, inspection, expansion, and stop decisions.

01 SOURCE ANALYSIS

Babel parser

Extract functions, direct awaits, calls, guards, literals and exact source locations.

02 CANDIDATE SEARCH

BM25 + vectors

Use cached lexical postings and CPU vector rank fusion to retrieve likely functions.

03 EVIDENCE CHECK

Bounded agent

Inspect supported, contradicted, or uncertain patterns; expand when needed.

04 REVIEWABLE OUTPUT

Node.js + browser

Stream the real decision trace, show source, and follow linked helpers.

Default: offline feature hashing • Optional: BGE / Jina Code / MiniLM q8

LIVE AGENT TRACE



A discovery becomes a saved rule.

Pin “awaits policyCheck” and follow the same function through its linked history.

FROM DISCOVERY TO A REPEATABLE CHECK

Behavior Watch

Pin a result below. Follow one behavior through renames, regressions, and restorations.

awaits policyCheck

Anchor: v2 · pairDevice

Parsed checks: direct await of policyCheck

Regression: v1 → v2 · Restoration: v2 → v3

v1 · Holds

connectDevice

► Evidence

v2 · Broken

pairDevice

Inferred rename / move

► Evidence

v3 · Holds

pairDevice

► Evidence

Checked 29/09/2026, 16:54:56 · indexed snapshots only

[Recheck](#) [Export evidence](#) [Remove](#)

Static evidence, not runtime proof. Saved in this browser; checks run on demand against the loaded index.

One pinned lineage

The check stays attached to the function through supported links.

Uncertainty stays visible

Ambiguous history and unsupported syntax remain unverified.

Evidence you can export

Save source, versions, parsed checks, and lineage confidence as JSON.

A saved rule tells a longer story.

The same static question is checked across linked snapshots, with gaps kept visible.

v1

HOLDS

The direct await is present in the source.

v2

BROKEN

The pinned static rule is contradicted.

v3

HOLDS

The direct await is present again.

Unknown and unlinked are first-class outcomes.

Ambiguous gaps stay unresolved. Recheck on demand and export the source-backed report.

Less history to reconstruct by hand.

Designed for reviewers investigating a specific behavior change.

01

Refactor review

Follow a function after a rename or move.

Inspect whether its await pattern changed.

02

Regression triage

Locate a changed guard or invocation order.

Compare the neighboring source version.

03

Version migration

Pin a rule before moving versions.

Recheck it across the loaded history.

DISCOVER → INSPECT → PIN A RULE → RECHECK

Three evaluations. Three different claims.

Measured results are presented with their scope and raw artifacts.

CONTROLLED VERSIONS

8 / 8

Correct top result

25 demo snippets + 1,000 distractors.
Synthetic development queries.

REAL REPOSITORIES

0.67 / 0.86

NDCG@10 · Async / Express

BGE hybrid; 60 source-reviewed,
AI-assisted labels.

FULL APPSRETRIEVAL

0.1571

NDCG@10 · Jina Code q8

3,765 queries / 8,765 Python items.
Independent encoder evaluation.

Full-test BGE baseline: 0.0505 NDCG@10 | Jina MRR@10: 0.1337

Limits: JS/JSX analysis • restricted query phrasing • heuristic lineage • no runtime proof.

AppsRetrieval does not evaluate version reranking. Scores across these sets are not comparable.

Three tests answer three different questions.

The strongest claim is specific: inspectable version discrimination in a controlled example.

Controlled history

8 / 8 top-1

Does evidence reranking choose the intended version in development queries?

Pinned real repos

0.6717 / 0.8577

Can BGE hybrid retrieve source-reviewed targets in Async / Express?

Official AppsRetrieval

0.15711 NDCG@10

How does independent Jina code encoding rank Python task solutions?

Do not combine these scores or describe static evidence as runtime proof.

The next evidence to earn.

A focused roadmap from the current prototype to broader use.

01

Independent evaluation

Human-labelled questions across real multi-commit histories.

Test rename ambiguity and missed links.

02

Broader code coverage

Extend parsing beyond JavaScript / JSX.

Improve paraphrase matching and supported static patterns.

03

Measured scale

Benchmark larger histories and repositories.

Add approximate retrieval only when justified.

Current boundary: bounded static analysis, local indexes, and explicit uncertainty.

Built, tested, and ready to demonstrate.

Published commit 0d23fdc • verification completed 29 September 2026

92

Node tests passed

9

Browser checks passed

CI: Linux + Windows + Docker
Chromium + Firefox + mobile
Python / Node encoder bridge

github.com/AdvayV/AgenticIntelligence

Working prototype + public repository

READY

README + reproducible setup

READY

Presentation + measured results

READY

Demo video, maximum 5 minutes

LINK PENDING

Find the change. Explain why. Keep watch.

CodeStrata makes code behavior
reviewable across versions.

github.com/AdvayV/AgenticIntelligence

Advay Vivek + Kaavish Gogia / Team CodeStrata

QUESTION



EVIDENCE



WATCH